

Neural Networks with Templates

- **Templates:** Example lightcurves created using Fourier coefficients to approximate real lightcurves
- **General Idea:** We can compare new input lightcurves to template lightcurves using a neural network to see how our input lightcurves (often with limited data) match up to complete lightcurves that we would expect from complete data
- **Approaches**
 - **Approach 1:** When taking in a new lightcurve as input, we can try to match it to a large number of template lightcurves and see which templates fit it best. We can then use these templates to calculate values like period, amplitude, color, and fourier parameters and then pass these values in as arguments into a neural network
 - **Pros**
 - Fairly interpretable features
 - We can more easily tell why the model is classifying something as it does
 - Doesn't require a large amount of training data
 - Does well with irregularly sampled data
 - Does well when working with multiple different sources that sample data in slightly different ways (e.g. Gaia, DES, Rubin, etc)
 - **Cons**
 - Model assumes a certain lightcurve shape and will struggle with unusual lightcurves and new subclasses
 - Extra step
 - Adds the step of template fitting
 - **Approach 2:** The templates can be directly used to regularize/train a neural network and then this neural network can be used to new light curves
 - **Pros**
 - Good at capturing complex patterns
 - Can generate lots of realistic examples to more accurately represent class distribution
 - **Cons**
 - Not very interpretable
 - Kind of just a black box
 - Needs a lot more data than just the templates
 - For good classification, best to have thousands of labeled examples
 - If a lot of labeled examples aren't provided, will be prone to overfitting
 - May struggle with irregularly sampled data
 - Neural networks sometimes often assume regular sampling
 - May be fixable by using specific models that handle irregular sampling well?

UMAP

- **General Idea:** UMAP is a dimensionality reduction algorithm (like PCA) but designed with the main goal of preserving local and global structure in data
 - We can think of UMAP as a way to project high-dimensional data into a 2D/3D space so that it can be better visualized
 - In other words, we can't easily graph high-dimensional data in a way that makes sense to the eye. UMAP aims to give us the general idea of the groupings of data in high dimensionality without actually being projected into a high-dimensional space
- **How it Works:** UMAP looks for the nearest neighbors of each datapoint and builds a graph in 2D/3D space where these neighbors in high-dimensional space are still neighbors in 2D/3D space
- **Pros**
 - Great for visualization
 - Preserves clusters extremely well and makes it super easy to tell what sort of data points are similar to each other
 - Can be used as input for machine learning algorithms
 - Works best for clustering algorithms like k-means, DBSCAN, GMM
- **Cons**
 - Graph coordinates are abstract and have no correlation to any input features
 - There is no way to tell what features are responsible for what in terms of separation
 - In other words, it's very difficult to actually interpret the graph